

RTFF: Random-to-Target Fabric Flattening Policy using Dual-Arm Manipulator

Kai Tang^{1,y}, Dipankar Bhattacharya^{2,y}, Hang Xu³, Fuyuki Tokuda⁴,
Norman C. Tien¹, and Kazuhiro Kosuge⁵

Abstract—Robotic fabric manipulation remains challenging due to fabric deformability and occlusions from wrinkles and the manipulator. This paper defines Random-to-Target Fabric Flattening (RTFF) as the task of bringing a randomly wrinkled fabric to an arbitrary user-specified wrinkle-free target pose. RTFF requires simultaneous flattening and pose alignment, where the two objectives are inherently coupled since flattening the fabric displaces its pose, while realigning it tends to introduce wrinkles. To solve this task, this paper anchors both the current and target fabric states to the same template mesh, enabling direct vertex-level wrinkle and pose assessment without registration. Building on this representation, a hybrid Imitation Learning–Visual Servoing (IL–VS) RTFF policy is proposed. A novel Mesh Action Chunking Transformer (MACT) leverages structured mesh observations to achieve goal-conditioned coarse alignment from a compact demonstration set, after which VS ensures precise convergence to the target. The policy is validated on a real dual-arm teleoperation system, demonstrating precise alignment to unseen target poses, fabric types, and scales. Code and videos: https://kaitang98.github.io/RTFF_Policy/

I. INTRODUCTION

Robotic manipulation of fabrics is a promising approach to reduce labor and enhance efficiency in garment production. A key task is sewing, where the same fabric panel must be reloaded in different positions to stitch seams, and random wrinkles are inherently introduced during sewing, loading, and repositioning [1]. To ensure accurate seam placement, fabric needs to be manipulated from an *initial random wrinkled state* to a *wrinkle-free, flattened target state*, whose pose (orientation and translation) varies across processes. This task, referred to as *Random-to-Target Fabric Flattening (RTFF)* (Fig. 1), requires iteratively selecting grasping points, simultaneously flattening local wrinkles, adjusting pose, and aligning the fabric until the target configuration is reached [2]. Efficient RTFF is also important in fabric cutting, screen printing, and ironing, where flatness and precise alignment determine garment quality [1], [3].

Fig. 1: Random-to-Target Fabric Flattening (RTFF) task.

Concretely, RTFF is a simultaneous flattening and pose-aligning problem, where the two objectives are *inherently coupled* since flattening the fabric displaces its pose, while realigning it tends to introduce wrinkles. It must satisfy the following three properties:

- (P1) Arbitrary target flat pose.** The policy must accept a *user-specified*, arbitrary flat pose (position and orientation), rather than a fixed canonical pose.
- (P2) Simultaneous flattening and pose alignment.** The policy must *jointly* remove all wrinkles across the entire fabric panel while aligning the panel to the target pose in a single unified framework.
- (P3) Wrinkle-aware state estimation.** The policy must have real-time access to *local* wrinkle states for flattening and the *global* fabric pose for alignment.

While prior work has made progress on related problems, such as fabric state estimation, flattening, and pose alignment, no existing method simultaneously satisfies all three requirements to achieve RTFF. Flattening and folding methods [4]–[6] ignore target pose (P1) or provide only global feedback (P3); alignment methods [1], [7], [8] assume a wrinkle-free start with given manipulation points (P2); and 3D state estimation methods [9]–[12] fit each observation independently and cannot compare directly to a target pose without a registration step (P1, P2, P3). A detailed review of related work is provided in Sec. II. In summary, the major limitation of existing methods is the lack of shared geometric reference between the observed fabric and the target pose.

Hence, this work proposes to define both the current fabric state and the target state with the same *template mesh*, derived from the fabric’s CAD model that deforms to match any observed state while preserving vertex identity.

^yThese authors contributed equally to this work.

¹Kai Tang and Norman C. Tien are with the Department of Electrical and Electronic Engineering, Faculty of Engineering, The University of Hong Kong, Hong Kong SAR, China. (E-mail: k.tang98@outlook.com)

²Dipankar Bhattacharya is with the Dyson School of Design Engineering, Imperial College London, London, United Kingdom.

³Hang Xu is with JD.COM, Shenzhen, China.

⁴Fuyuki Tokuda is with the Unprecedented-scale Data Analytics Center, Tohoku University, Sendai 980-0845, Japan, and also with the Graduate School of Information Sciences, Tohoku University, Sendai 980-0845, Japan.

⁵Kazuhiro Kosuge is with the Department of Mechanical Engineering, City University of Hong Kong, Hong Kong SAR, China.

Prior work [13] applied template meshes for state estimation. Building on this formulation, this paper proposes a template mesh-based hybrid IL–VS RTFF policy (P2): (i) an Imitation Learning (IL) stage that learns from a small demonstration set to perform coarse alignment and wrinkle reduction guided by the mesh state, (ii) followed by a Visual Servoing (VS) stage that activates once the mesh signals sufficient flatness and drives the fabric to precise alignment with the target. The contribution of this paper is summarized as follows:

A template mesh-based RTFF formulation that anchors both the current and target fabric states to the same canonical mesh, enabling direct vertex-level wrinkle and pose assessment (P1, P3), and a hybrid IL–VS framework that enables simultaneous fabric flattening and pose alignment (P2).

A goal-conditioned IL policy, Mesh Action Chunking Transformer (MACT), that leverages the template mesh state representation to achieve RTFF from a compact demonstration set.

Physical validation on a dual-arm teleoperation system, demonstrating precise alignment to unseen target configurations and robustness across different fabric types and scales.

II. RELATED WORK

A. Fabric flattening and folding

Pick-and-place policies trained from visual observations learn to reduce wrinkles by maximizing a global flatness metric such as image coverage [4]. Dynamic folding controllers [5], [10] use reinforcement learning with a global shape reward to execute pre-defined fold targets. While effective at these tasks, these methods do not accept a user-specified arbitrary target pose (P1) and cannot ensure locally wrinkle-free (P3), since their objectives use global/sparse rewards rather than local wrinkle state.

B. Fabric alignment and VS

VS methods reposition fabric toward a goal configuration using Jacobian-based control with a rigidity-preserving lattice or mesh model [8], [14] or Transformer-predicted texture-match error [7]. Both require the fabric to be approximately flat or held mechanically taut. However, wrinkles in RTFF break the near-rigid deformation model and the gripper-tension assumption on which these methods rely (P2). CNN-based servoing [6] does address simultaneous positioning and flattening (P2), but it converges to a single fixed target baked into the CNN weights, hence it cannot accept an arbitrary user-specified pose at inference time (P1), and provides no per-region wrinkle feedback (P3).

C. Fabric state estimation

Fabric state has been represented using pixels [6], [7], edge features [15], and latent vectors [16], but these 2D representations provide no 3D geometric structure and cannot assess wrinkle severity or pose offset relative to a target. Richer geometric representations improve reconstruction fidelity: point-cloud tracking [9] and per-step mesh fitting [10],

[11] reconstruct cloth deformation from depth or RGB-D inputs, while Gaussian splatting [12] further improves photometric accuracy. However, these methods estimate each state independently, without persistent vertex correspondence shared with the target, so comparing an observation to a user-specified goal (P1) requires a separate registration step, and there is no direct signal for simultaneously decomposing local wrinkle severity and pose offset (P2, P3). Template-based tracking [13] introduces persistent vertex correspondence by continuously deforming a canonical mesh; this work extends that representation to also define the target state, unlocking direct per-vertex error computation without any registration.

D. IL for manipulation

Teleoperation-based IL avoids explicit dynamics modeling and sim-to-real gaps by learning directly from expert demonstrations. Action Chunking Transformers (ACT) [17], [18] condition a Transformer on image observations to predict chunked action sequences, while diffusion-based policies [19], [20] generate multimodal action distributions. These methods typically rely on RGB or RGB-D observations and large demonstration datasets, limiting alignment precision and generalization to novel target configurations in RTFF. This work builds on ACT [17] but replaces image inputs with mesh vertex positions and real-time alignment error, enabling goal-conditioned RTFF from a compact demonstration set and making the policy directly sensitive to 3D wrinkle geometry and pose offset (P2, P3).

III. PROPOSED FRAMEWORK OUTLINE

A. Robotic manipulation platform

The robotic manipulation platform (Fig. 2 (b)) consists of two 6-DoF manipulators suspended from an aluminum frame, each with a wrist-mounted F/T sensor and a custom end-effector. Dual-arm impedance control [1] enables safe interaction with the environment without joint torque sensing. An RGB-D sensor on top provides images for real-time state estimation. Human demonstration data for RTFF is collected via teleoperation using two SpaceMouse devices to command end-effector velocities and desired forces. A low-level workstation provides high-frequency control of the manipulators, while a high-level computer runs state estimation and RTFF policy modules at a lower frequency.

For operating the platform, a world frame $\mathbf{w}$ (Fig. 2 (b)) is defined with the z-axis normal to the table and all vectors are expressed in this frame. At time t , the proprioceptive state of end-effector $i \in \{L, R\}$ (L left, R right) is $\mathbf{m}_{t,i} \in \mathbb{R}^{12}$, containing its 6D pose and wrench; the combined state is $\mathbf{m}_t = [\mathbf{m}_{t,L}^\top; \mathbf{m}_{t,R}^\top]^\top \in \mathbb{R}^{24}$. The action of the manipulator i is $\mathbf{a}_{t,i} = [v_{x,i}; v_{y,i}; \dot{\theta}_{z,i}; F_{z,i}^d]^\top$, with $v_{x,i}; v_{y,i}$ planar velocities, $\dot{\theta}_{z,i}$ yaw rate, and $F_{z,i}^d$ the desired vertical force for impedance control. The joint action $\mathbf{a}_t = [\mathbf{a}_{t,L}^\top; \mathbf{a}_{t,R}^\top]^\top \in \mathbb{R}^8$ drives the end-effectors for fabric manipulation.

B. RTFF task description

Let the canonical template mesh be $T = (V^{\text{TPL}}; E^{\text{TPL}})$, a standardized flattened fabric reference derived from the

Fig. 2: Overview of the proposed RTFF policy framework.

fabric CAD model. Here, $V^{\text{TPL}} = f v_i^{\text{TPL}} g_{i=1}^N$ are vertex positions, and $E^{\text{TPL}} = f(i; j) \mid j \in \mathcal{N}_i, j \in E^{\text{TPL}} \mid j = N_e$, are triangular mesh that connects the vertices. At time t , the RGB-D input and T are used for state estimation to predict the deformed mesh $\hat{M}_t = (\hat{V}_t; E^{\text{TPL}})$, with $\hat{V}_t = f \hat{v}_{i;t} g_{i=1}^N$. Reusing E^{TPL} preserves vertex connectivity. Given a wrinkle-free, flattened target mesh derived from the template mesh (red, Fig. 1), $M = (V; E^{\text{TPL}})$; with $V = f v_i g_{i=1}^N$ encoding the user-specified target pose (P1), the *mean alignment error* between the predicted mesh $\hat{M}_t$ and the target mesh M at time t along $x; y; z$ is

$$e_t = \frac{1}{N} \sum_{i=1}^N e_{i;t}; \quad \text{where } e_{i;t} = \hat{v}_{i;t} - v_i \in \mathbb{R}^3; \quad (1)$$

Here, $e_t \in \mathbb{R}^3$, and $e_{i;t}$ denotes the i^{th} vertex alignment error, providing the per-vertex wrinkle and pose feedback required by (P3). The template mesh T , shared by $\hat{M}_t$ and M , ensures correspondence-preserving state estimation, deformation tracking, and sim-to-real transfer. **Note:** Throughout the paper, by a slight abuse of notation, M (such as $M; \hat{M}$) denotes the vertex position V (such as $V; \hat{V}$).

Definition 1 (RTFF task). Assuming fabric wrinkles do not involve irreversible deformations (such as stretched fibers or permanent creases) and that the fabric can be fully flattened on the table with a sufficient sequence of dual-arm actions $\mathbf{a}_{0:T-1} = f \mathbf{a}_0; \dots; \mathbf{a}_{T-1} g$, the RTFF task is formulated as minimizing the terminal mean alignment magnitude (terminal mean alignment norm) over $\mathbf{a}_{0:T-1}$

$$\min_{\mathbf{a}_{0:T-1}} e_T(M_T; M); \quad (2)$$

where T denotes the planning horizon. Equation (2) evaluates the final state after $\mathbf{a}_{0:T-1}$ transforms the initial wrinkled mesh $\hat{M}_0$ into M .

C. RTFF policy

The key challenge in RTFF is wrinkled fabric state estimation under occluded RGB-D from the manipulator end-

effectors, while tracking mesh states to plan manipulator actions. This section first introduces state estimation to handle occlusions, followed by the policy.

1) *Fabric state estimation:* The fabric state estimation of RTFF policy predicts the fabric mesh (green, Fig. 1). The current RGB image and point cloud are first processed by a fabric depth rendering module to produce segmented point cloud Q_t^{SEG} , and segmented fabric depth image D_t^{SEG} at time t . Next, D_t^{SEG} and T are input to a Graph Attenuation Network (GAT), $G(\cdot)$, trained on synthetic data from a *triangular mass-spring mesh model with occlusions*, to predict the deformed fabric mesh $\hat{M}_t^{\text{GAT}}$ in real time. However, to ensure smoother mesh-prediction transitions between consecutive time steps of the manipulation policy, and further improve accuracy, sim-to-real fine-tuning is performed, in which $\hat{M}_t^{\text{GAT}}$ is refined by a residual correction $\hat{M} = \hat{M}_t^{\text{GAT}}; Q^{\text{SEG}}$ by optimizing a one-sided Chamfer loss that aligns the mesh with Q^{SEG} . The final prediction can be written as

$$\hat{M}_t = \hat{M}_t^{\text{GAT}} + \hat{M}_t(\cdot); \quad \text{with } \hat{M}_t^{\text{GAT}} = G(D_t^{\text{SEG}}; T); \quad (3)$$

2) *Fabric manipulation policy:* Given the prediction from (3), fabric manipulation is formulated as a Markov Decision Process (MDP) $M = (S; A; P)$, where the observation state at time t is $s_t = f M; \hat{M}_t; D_t; E_t; m_t g \in S$, where D_t is the environment depth image and *alignment error* $E_t = f e_{i;t} g_{i=1}^N$. Given s_t and action $\mathbf{a}_t \in A$, transition $P(s_{t+1} \mid s_t; \mathbf{a}_t)$ captures nonlinear fabric dynamics.

To solve the RTFF task, a hybrid manipulation policy $\pi: S \rightarrow A$ is proposed, consisting of an IL policy π^{IL} for coarse alignment and a VS policy π^{VS} for fine alignment, jointly addressing simultaneous flattening and pose alignment (P2). Given state $s_t^{\text{IL}} = s_t$, the IL policy is modeled as the conditional distribution $\pi^{\text{IL}}(\mathbf{a}_t \mid s_t^{\text{IL}})$; which is fit implicitly from human demonstrations. The learned policy generates sequence of manipulation actions $\mathbf{a}_t$ such as grasp point selection, fabric flattening, pose adjustment, and coarse alignment to M . For the VS policy, the reduced state $s_t^{\text{VS}} = f M; \hat{M}_t; E_t; m_t g$ is used to design a closed-loop

control law $^{VS}(s_t^{VS})$ which coordinates dual-arm actions for fine alignment. Note that P is implicit in IL (learned from demonstrations) and explicit in VS (defined by Jacobian).

Definition 2 (RTFF policy). The hybrid policy is given by

$$a_t = \begin{cases} ^{VS}; & \text{if } t \leq T; \\ ^{IL}; & \text{otherwise;} \end{cases} \quad (4)$$

where $t \in [0; 1]$ is flatness, fraction of mesh vertices on tabletop within a height tolerance; $ke_t k_2$ is the alignment error magnitude (mean alignment error norm); and $F_{z,i;t}$ is the end-effector vertical force with $i \in [1; R]$. The IL policy first performs coarse alignment to reduce wrinkles and bring the fabric closer to M . When the fabric is flat and aligned, $t \leq T$ and $ke_t k_2 \leq \epsilon$, IL selects grasping points for handoff. After both arms grasp the fabric, $F_{z,i;t} = F_z^g$, control switches to VS for fine alignment, producing the action sequence $a_{0:T-1}$ that minimizes (2).

3) *MACT*: This work implements IL with a novel MACT policy conditioned on the predicted mesh $\hat{M}$ from (3), the target mesh M , and the alignment error E_t using a Transformer backbone. The policy outputs action chunks [17] for efficient storage and execution while capturing *non-Markovian* behaviors in demonstrations. Given an observation s_t^{IL} , MACT predicts a horizon of N_h actions $\hat{a}_{t:t+N_h}$. Hence the one-step IL policy $^{IL}(a_t | j | s_t^{IL})$ in (4) can be extended and rewritten as $^{IL}(\hat{a}_{t:t+N_h} | j | s_t^{IL})$. At deployment, this horizon is executed in a receding manner via temporal ensembling: overlapping predictions are averaged to yield the next action, thereby recovering $^{IL}(a_t | j | s_t^{IL})$. Finally, MACT supervision is provided by a reconstruction loss, computed as the ℓ_1 discrepancy between $\hat{a}_{t:t+N_h}$ and the ground-truth chunk $a_{t:t+N_h}$ from demonstrations, averaged across N_h .

IV. METHODOLOGY

A. State estimation implementation

1) *Synthetic data generation*: A fabric state is modeled as a triangular mass-spring mesh $M^{SIM} = (V^{SIM}; E^{TPL})$; where $V^{SIM} = f v_i^{SIM} g_{i=1}^N$ are 3D vertex positions and E^{TPL} connects neighboring vertices as point masses. Each edge $(i; j) \in E^{TPL}$ acts as a spring with vector $d_{ij}^{SIM} = v_j^{SIM} - v_i^{SIM}$; $kd_{ij}^{SIM} k_2$; enforcing structural constraints. This mesh is implemented in *Blender* [21]. To generate diverse states, randomized manipulations were applied, including multi-hand and single-hand interactions (with or without lifting), as well as compression and shearing. After each manipulation, the final deformed mesh M^{SIM} is saved as the simulation output.

After mesh generation, depth images are rendered with *PyTorch3D* [22]. Centralized meshes are batched, processed with a custom depth shader, and z-buffer values normalized to a fixed range. The outputs are converted to single-channel grayscale depth images, and the resulting D^{SIM} from M^{SIM} are saved for GAT training. Finally, each D^{SIM} is converted to a simulated point cloud $Q^{SIM} = f q_j^{SIM} g_{j=1}^{N_q}$; by mapping each pixel to a 3D point.

2) *Mesh prediction model*: The model follows a GAT $G(\cdot)$ framework as discussed in [13], supervised on synthetic datasets (Sec. IV-A.1). The objective is

$$L_{GAT-Train} = L_{Vtx} + \alpha L_{Key} + \beta L_{Cham}; \quad (5)$$

The vertex loss measures ℓ_1 distance between predicted vertices $\hat{v}_i^{GAT}$ and Ground Truth (GT) v_i^{SIM} , while the key-point loss applies the same penalty on semantic keypoints (neckline, shoulder, arm, hem)

$$L_{Vtx} = \frac{1}{N} \sum_{i=1}^N \|v_i^{GAT} - v_i^{SIM}\|_1; \quad L_{Key} = \frac{1}{N_k} \sum_{i=1}^{N_k} \|v_i^{GAT} - v_i^{SIM}\|_1; \quad (6)$$

where $v_i = \hat{v}_i^{GAT} - v_i^{SIM}$, N_k denotes number of keypoints. To refine boundary and surface similarity, the Chamfer loss measures distances from the simulated point cloud Q^{SIM} to predicted vertices

$$L_{Cham} = \frac{1}{N_q^{SIM}} \sum_{j=1}^{N_q^{SIM}} \min_{i \in Q^{SIM}} \|q_j^{SIM} - \hat{v}_i^{GAT}\|_1; \quad (7)$$

Finally, $G(\cdot)$ is trained by minimizing (5) with meshes M^{SIM} as ground truth and depth images D^{SIM} as input.

3) *Real fabric depth rendering*: At time t , the segmented depth image D_t^{SEG} is produced by a GPU-accelerated pipeline. RGB and point cloud data are first captured with the RealSense sensor (Sec. III-A). SAM2 [23] segments the RGB frame to obtain a fabric mask, which is applied to the point cloud using Torch functions to isolate fabric points. The resulting cloud is denoised and centralized, yielding $Q_t^{SEG} = f q_j^{SEG} g_{j=1}^{N_q^{SEG}}$; where N_q^{SEG} is the number of points. Finally, Q_t^{SEG} is rasterized with PyTorch3D to form D_t^{SEG} for fabric state estimation.

4) *Sim-to-real fine-tuning*: To obtain the residual correction $\hat{M} = \hat{M}^{GAT} + \hat{M}^{SEG}$ defined in Sec. III-C.1, a one-sided Chamfer loss that aligns the GAT predicted mesh $\hat{M}^{GAT}$ with the segmented point cloud Q^{SEG} is written as

$$L_{Cham-R} = \frac{1}{N_q^{SEG}} \sum_{j=1}^{N_q^{SEG}} \min_{i \in Q^{SEG}} \|q_j^{SEG} - \hat{v}_i^{GAT}\|_1; \quad (8)$$

To avoid vertex drift, an edge-length regularization preserves local mesh structure is defined as

$$L_{Edge-R} = \frac{1}{N_e} \sum_{(i;j) \in E^{TPL}} \|d_{ij}^{GAT} - d_{ij}^{TPL}\|_1; \quad (9)$$

The final objective is

$$L_R = \alpha L_{Cham-R} + \beta L_{Edge-R}; \quad (10)$$

where weights α and β balance alignment and regularity. Minimization of L_R provides the residual correction $\hat{M}$ to the GAT prediction, defined as $\hat{M} = \arg \min_M L_R(\hat{M}^{GAT} + M; Q^{SEG})$; which yields the final predicted mesh (3).

B. Hybrid manipulation policy implementation

1) *Mesh Action Chunking Transformer (MACT)*: The MACT policy (Sec. III-C.2, Fig. 3) takes as input the raw environment depth image $D_t \in \mathbb{R}^{H \times W \times 1}$, which is encoded by a ResNet-18 backbone into a feature map of size $\mathbb{R}^{H^0 \times W^0 \times d_{model}}$, where d_{model} is a predefined model dimension. The feature is then flattened along the spatial dimension to

Fig. 3: MACT policy architecture.

obtain a $(H^0 W^0) \ d_{\text{model}}$ token sequence. To preserve spatial information, a 2D sinusoidal positional embedding is added to the feature sequence. Additional inputs include: (i) the target mesh V with alignment error, $M = E_t \in \mathbb{R}^{N \times 6}$, embedded to $N \ d_{\text{model}}$; (ii) the predicted mesh vertices $\hat{V} \in \mathbb{R}^{N \times 3}$, embedded to $N \ d_{\text{model}}$; (iii) the EE states $\mathbf{m}_t = [\mathbf{m}_{t,L}^>; \mathbf{m}_{t,R}^>]^> \in \mathbb{R}^{1 \times 24}$ (i.e., two manipulators, each with 6D position + 6D wrench), embedded to $1 \ d_{\text{model}}$. Additional sinusoidal positional embedding is added to these input sequences.

A style variable $z \in \mathbb{R}^{1 \times 24}$ is output by Transformer-based Conditional Variational Autoencoder (CVAE). It takes as input the EE states and a target action sequence of length N_h prepended with a learned “[CLS]” token, forming a $(N_h + 2)$ sequence. The “[CLS]” feature outputs the mean and variance of z , which is sampled for training and set to the prior mean (zero) at inference for deterministic decoding. z is projected to $1 \ d_{\text{model}}$ via linear layers, yielding the concatenated encoder input $\mathbf{s}_t^L \in \mathbb{R}^{(H^0 W^0 + 2N + 2) \ d_{\text{model}}}$.

The MACT decoder attends to encoder features via cross-attention, using a fixed $N_h \ d_{\text{model}}$ positional sequence as input. It outputs an $N_h \ d_{\text{model}}$ sequence, down-projected by an MLP to $N_h \times 8$, representing manipulator actions $\mathbf{a}_t = [\mathbf{a}_{t,L}^>; \mathbf{a}_{t,R}^>]^> \in \mathbb{R}^{2 \times 8}$ for the two manipulators over N_h steps. The model is trained with a VAE objective: reconstruction of action chunks plus a regularization term aligning the VAE encoder with a Gaussian prior.

2) *Visual Servoing (VS)*: At time t , the predicted mesh $\hat{M}_t = (\hat{V}_t; E^{\text{TPL}})$ meets the condition of $^{\text{VS}}$ in (4) to switch to VS. Assuming the fabric undergoes a rigid-body motion under dual-arm control [7], the mean alignment error between $\hat{M}_t$ and the target mesh $M = (V; E^{\text{TPL}})$ given by (1) is reformulated as an objective function

$$\min_{R_t \in \text{SO}(3); \mathbf{t}_t \in \mathbb{R}^3} \frac{1}{N} \sum_{i=1}^N (R_t \hat{\mathbf{v}}_{i;t} + \mathbf{t}_t - \mathbf{v}_i)^2; \quad (11)$$

where $\hat{\mathbf{v}}_{i;t} \in \mathbb{R}^3$, $\mathbf{v}_i \in V$ are corresponding vertices, and $T_t = (R_t; \mathbf{t}_t) \in \text{SE}(3)$ is the rigid-body transformation matrix w.r.t. w . Since motion is restricted to the tabletop, T_t reduces to planar translation $\mathbf{p}_t = [x_t; y_t]^>$ and rotation $\theta_t = \text{atan2}(R_t(2;1); R_t(1;1))$. The pose error vector is then $\mathbf{e}_t^p = (\mathbf{p}_t^>; \theta_t)$, where $\mathbf{p}_t$ is the residual translation, and θ_t is the rotation to align $\hat{M}_t$ with M .

To eliminate this $\mathbf{e}_t^p$, a coordinated motion of the dual-arm manipulator is calculated by introducing position-based VS and rigid-body motion. The control motion is a planar spatial twist, $\mathbf{a}_t^{\text{vs}} = [v_x; v_y; \theta]^> - \mathbf{a}_t$, defined in Sec. III-A, and F_z^d is set as a constant. In position-based VS, the error dynamics are written as $\dot{\mathbf{e}}_t^p = L \mathbf{a}_t^{\text{vs}}$; where L is

Fig. 4: Simulated and real fabric mesh prediction.

the interaction matrix (Jacobian) by assuming a rigid body motion at time t . By imposing exponential decay of the error as $\dot{\mathbf{e}}_t^p = -\gamma \mathbf{e}_t^p$ with a gain $\gamma \in \mathbb{R}$, the control input becomes

$$\mathbf{a}_t^{\text{vs}} = L^\gamma \mathbf{e}_t^p = -L^\gamma \mathbf{e}_t^p; \quad (12)$$

where L^γ is the Moore–Penrose pseudo-inverse of L . The control input is calculated at each iteration, providing real-time feedback that mitigates the impact of rigid-body assumptions and increases alignment precision [14].

V. RESULTS

A. Hardware specifications

The manipulators in the proposed platform (Fig. 2 (b)) are DENSO VS-068 arms, each with an ATI Axia80-M8 F/T sensor and a custom 3D-printed end-effector. Teleoperation uses two SpaceMouse[®] Compact devices. The vision system is an Intel[®] RealSense[™] LiDAR Camera L515. The lower-level workstation is powered by an Intel[®] Xeon[®] W-2295 CPU running INtime OS over EtherCAT at 4 kHz, and the high-level computer is equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel[®] Core[™] i9-14900KF CPU, and 128 GB RAM running Ubuntu 24.04 at 20 Hz.

B. State estimation results

This section presents state estimation results on a *T-shirt front panel* fabric with dimensions 400 × 550 mm (Fig. 4 (e)). The canonical template mesh T , with $N = 562$ vertices (Fig. 4 (a)), is generated in MeshLab [24].

Synthetic data generation: For synthetic data generation in Blender, fabric simulation parameters were manually tuned for real fabric behavior. Random perturbations are applied during data generation to account for uncertain fabric parameters and mitigate the sim-to-real gap. Rendered cloth example in Blender is shown in Fig. 4 (b). Finally, to generate simulated segmented depth image Fig. 4 (c) for GAT training, the simulated fabric meshes M_t are saved and processed through the same depth rendering pipeline (Sec. IV-A.3) as real data to ensure consistency (Fig. 4 (c)).

Training mesh prediction model: About 60,000 simulated depth–mesh pairs were generated and split into 80% training, 10% validation, and 10% testing to train the GAT (Sec. IV-A.2) for wrinkle-aware state estimation (P3) in the RTFF policy. Training used 512 × 512 depth inputs, batch size 16, 500 epochs, and learning rate 1×10^{-4} ; the loss weights in (5) were $w_k = 1$ and $w_c = 0.5$. Data augmentation

Fig. 5: Evaluation results with controlled occlusion.

included: (i) additive Gaussian noise; (ii) random rigid-body rotations; (iii) synthetic occlusions of various shapes and sizes to mimic EE and manipulator (external) occlusions; and (iv) pixel-level z-variations to capture wrinkles. For sim-to-real, the mesh $\hat{M}^{\text{GAT}}$ is predicted from segmented depth D^{SEG} using the trained GAT $G(\cdot)$, then refined in real time by optimizing the one-sided Chamfer loss (8) with Adam using $\text{Cham-R} = 1$, $\text{Edge-R} = 0.01$, learning rate 2×10^{-3} , and five steps per observation to yield the final predicted mesh $\hat{M}$.

Evaluation: State estimation is first shown in simulation, where depth (Fig. 4 (c)) is applied to the trained GAT to predict meshes that recover fabric state (Fig. 4 (d)). For evaluation, the trained GAT was tested on a held-out test set under synthetically masked occlusion, reporting *Mean Vertex Error (MVE)* and *Mean Keypoint Error (MKE)* in Fig. 5. Here, MVE and MKE are the mean Euclidean distances between predicted and GT mesh vertices and semantic keypoints (Sec. IV-A.2), respectively. The network maintains high accuracy under moderate occlusion (MVE = 0.0070 m at 20%, 0.0111 m at 40%), but errors accumulate steeply beyond 40%, establishing a practical guideline to keep occlusion below 40%.

In real-world experiments, regions missing due to EE occlusion (Fig. 4(f)–(g), red circle) are inferred using the GAT prediction, with Chamfer loss-based refinement improving accuracy (Fig. 4(h)). For evaluation, where GT is unavailable, *One-sided Chamfer Distance (OCD)* from the predicted mesh to the detected depth image was measured under controlled occlusion. Initial predictions showed slightly higher OCD than simulation, but refinement reduced OCD by 21–31% across all occlusion levels, aligning with simulation performance and confirming accurate mesh prediction in the nominal regime.

C. Manipulation policy results

This section demonstrates the performance of the proposed RTFF manipulation policy pipeline. The policy is built on the LeRobot framework [25] and trained with human demonstrations collected on the robotic platform discussed in Sec. III-A. The system runs across several parallel threads: the main MACT control thread at 20 Hz and the mesh prediction thread at 10 Hz, where SAM2 segmentation, depth rendering, and GAT forward pass account for 30%, 35%, and 20% of computation, respectively.

During training, demonstrations are collected using a fabric panel of thickness 0.32 mm, areal density 0.21 kg/m², and Young’s modulus 21.81 MPa. Because fabric placement and wrinkle formation are inherently stochastic on this plat-

form, identical states cannot be reproduced. Thus, the policy must be strictly observation-driven for localizing wrinkles, planning motions to flatten and rotate the fabric to the target (P1, P2), selecting grasping points before switching to VS using the current and target meshes (P3), and coordinating collision-free dual-arm actions.

Finally, the data collection and evaluation process on this platform proceeds as follows: (i) the dual-arm manipulator is initialized to a home pose, and a wrinkled fabric is placed on the table; (ii) a user-specified target state M (P1) is randomly generated within the camera’s field of view; (iii) demonstrations are collected by manually controlling the manipulators with a SpaceMouse to perform RTFF across multiple episodes, restricted to a single fabric type with fixed material and scale; (iv) these demonstrations are used to train the MACT component of the RTFF policy; (v) the trained policy is executed on fabrics with previously unseen wrinkles, thickness, textures, materials, and scales to jointly remove all wrinkles and flatten them (P2) toward the target state; (vi) finally, the manipulator returns to the home pose and experimental results are recorded for evaluation.

1) *Experimental protocol and evaluation metrics:* For experiments, a pre-defined set of fabric states and targets was needed, but consistent wrinkle reproduction was infeasible. To ensure fair comparison, 20 target meshes M were pre-sampled and used in all trials. Initial states were generated by orienting the neckline at 0°, 90°, 180°, and 270°, each with five trials introducing random wrinkles, yielding 20 states matching the 20 targets. Experimental parameters were set to $e = 0.05$ m, $\alpha = 0.95$, and $F_z^g = 2.5$ N. These thresholds are empirically selected to ensure VS is triggered only when the fabric is sufficiently flat, the alignment error is within the convergence range of VS, and the fabric is securely grasped by both end-effectors. Switching too early risks unstable servoing on a partially wrinkled fabric, while switching too late wastes manipulation time unnecessarily.

The RTFF policy is evaluated using three metrics: (i) *terminal alignment error magnitude (terminal mean alignment error norm)* $ke_T k_2$, where e_T is given by (1) at $t = T$; (ii) *failure*, defined as $ke_T k_2 > 0.05$ m; and (iii) *Intersection over Union (IoU)* between the final mesh and target mesh M , measuring geometric overlap

$$\text{IoU} = \frac{jA_{\text{final}} \setminus A_M j}{jA_{\text{final}} \cup A_M j}, \quad (13)$$

where A_{final} and A_M denote the final and target mesh regions. An IoU of 1 with sufficiently small $ke_T k_2$ indicates perfect alignment, while values near 0 imply negligible overlap. IoU is computed only for non-failure trials, and experiments terminate early if the first five trials all fail.

2) *MACT training:* Human demonstrations trained the IL (MACT) part of the policy. In total, 50 episodes of 30–90 seconds each were collected at 20 Hz. To broaden the data distribution, random translations were applied to the fabric’s initial position. For orientation coverage, the fabric’s neckline was uniformly rotated on the table before wrinkles were introduced. To enhance robustness, online perturbations

Fig. 6: Example RTFF sequences with (top) and without (bottom) Visual Servoing (VS).

were introduced: (i) flattened fabric regions were randomly re-wrinkled, and (ii) random displacements from the correct grasping point were applied by the operator via direct end-effector impedance control (without the SpaceMouse).

The MACT component of the RTFF policy was trained with depth inputs (640 480), chunk size $N_h = 100$, and model dimension $d_{\text{model}} = 512$. Training used 100,000 steps, a batch size of 4, and a learning rate of $1 \cdot 10^{-5}$.

3) *RTFF policy evaluation*: Table I, Row 1, reports the full RTFF policy, showing stable convergence to an average error of about 0.01 m after 20 trials with low variance, outperforming all ablations.

TABLE I: Ablation study of RTFF policy.

Method	Terminal alignment error (m)		IoU		Fail.
	Mean	Std.	Mean	Std.	
Ours	0.011	0.002	0.926	0.018	-
w/o VS (i)	0.028	0.010	0.871	0.05	-
w/o $\hat{M}_t$ and E_t (ii)	-	-	-	-	5/5
w D_t^{SEG} , w/o $\hat{M}_t$ and E_t (iii)	-	-	-	-	5/5
Type	0.0145	0.004	0.911	0.031	-
Scale	0.021	0.004	0.893	0.022	-

During execution (Fig. 6, top row), the RTFF policy showed consistent behaviors: (i) adjusting the EE pose in the air and manipulating only after a successful grasp; (ii) grasping flat areas on both sides of wrinkles and removing them with opposing motions; (iii) releasing stretched fabric to restore shape before re-grasping; (iv) smoothly switching to VS once criteria given by (4) are met; and (v) switching back to MACT if VS creates new wrinkles and voids (4), removing them before resuming VS. This can arise from position–force control coupling effects that cause unexpected end-effector motion during VS execution. The results show that the policy achieves RTFF by learning fabric state and its relation to the target mesh. Using depth images with a mesh-based representation supports robust wrinkle extraction and accurate displacement estimation between current and target states. With real-time visual feedback, the policy converges reliably across diverse initial configurations and unseen targets.

4) *Ablation studies*: Ablations were performed on key RTFF components and MACT. The ablation conditions are: (i) *Without (w/o) VS*: The IL policy runs for up to 90 s per trial, after which the episode is terminated and outcomes recorded. (ii) *W/o current predicted fabric mesh $\hat{M}_t$ and alignment error E_t embeddings*: Retrain the MACT (Fig. 3)

without $\hat{M}_t$ and E_t embeddings, with the depth image D_t , robot states and the target mesh. (iii) *With (w) segmented fabric depth D_t^{SEG} , and w/o $\hat{M}_t$ and E_t* : Retrain the MACT with the D_t^{SEG} , D_t , robot states, and the target mesh as input.

Ablation study results: Rows 2–4 of Table I report the results, with main findings: (i) *W/o VS*: The policy avoids outright failures but shows larger errors and variance, reflecting unreliability (Fig. 6, bottom row). Near the target, it fails to extract salient cues, leading to stationary behavior or repeated grasping at the same spot without moving the fabric. (ii) *W/o $\hat{M}_t$ and E_t* , and (iii) *w D_t^{SEG}* : Both variants fail. Without the mesh-based state estimate and error signal, MACT lacks explicit correspondence between the current and desired fabric states. Replacing them with D_t^{SEG} provides geometrically ambiguous observations that the policy cannot exploit from limited demonstrations. In both cases, the policy occasionally flattens local wrinkles but fails to rotate or translate the fabric to match the target configuration.

These results verify the central role of the mesh in the hybrid IL–VS policy. The mesh representation enables sample-efficient IL from few demonstrations, strengthens IL by providing a structured link between the current fabric state and the target, allowing for a smooth switching between IL and VS, and yields a well-defined representation (P3) that enables VS to converge $\hat{M}_t$ to the given target (P1).

5) *Generalization experiments*: In garment production, fabrics differ in color, material, and size. To test policy generalization, two additional experiments were conducted: (i) *Type* — five fabrics of different materials (cotton and polyester) cut to the baseline shape, with thickness ranging from 0.23–0.40 mm, areal density from 0.14–0.23 kg/m², and Young’s Modulus from 12.43–21.81 MPa. (ii) *Scale* — the baseline fabric reduced to 80% size. Each experiment comprised 20 trials, with results given in Rows 5–6 of Table I and examples in Fig. 7.

The results show that the policy, trained solely on the baseline fabric, successfully aligns (P2) unseen fabric types and scales to desired user-specified target states (P1) without re-training. Although alignment accuracy moderately degrades relative to the baseline, the policy maintains functional performance across all tested configurations. Depth images with predicted mesh enable robust wrinkle features (P3) and accurate alignment error estimation between current and target states. Combined with real-time visual feedback, the policy consistently converges across diverse fabric initial states and

Fig. 7: Examples of fabric Types (a)–(c) and Scale (d).

user-specified target states, indicating strong generalization to varying fabric types and scales.

VI. CONCLUSION, LIMITATIONS AND FUTURE WORK

This work presents a Random-to-Target Fabric Flattening (RTFF) policy using a template mesh that anchors both the current fabric state and the user-specified target configuration to the same canonical mesh derived from the fabric's CAD model. This shared reference enables arbitrary target pose specification (P1), per-vertex wrinkle state and global pose feedback, and direct inter-mesh error computation without registration (P3). Building on this, a template-mesh two-stage RTFF policy is proposed (P2): a Mesh Action Chunking Transformer (MACT) Imitation Learning (IL) stage learns from a compact demonstration set for coarse alignment and wrinkle reduction guided by mesh state and alignment error, followed by a Visual Servoing (VS) stage that drives precise alignment to the target. Ablation studies confirm the central role of the mesh in the hybrid IL–VS policy, with each component necessary for reliable RTFF, and physical experiments demonstrate strong generalization to unseen target poses, fabric types, and scales.

Limitations and future work: Despite promising results, the framework has several limitations. The system cannot handle folded configurations that require 3D manipulation, and the current policy is shape-specific. Future work will extend the method to support multiple templates for different fabric geometries, improve the inference speed of fabric state estimation, and extend the RTFF policy to full 3D manipulation.

REFERENCES

- [1] K. Tang, X. Huang, A. Seino, F. Tokuda, A. Kobayashi, N. C. Tien, and K. Kosuge, "Fixture-free automated sewing system using dual-arm manipulator and high-speed fabric edge detection," *IEEE Robot. Autom. Lett.*, vol. 10, no. 9, pp. 8962–8969, 2025.
- [2] P. Taylor and D. Pollet, "Why is automated garment manufacture so difficult?" in *1997 8th International Conference on Advanced Robotics. Proceedings. ICAR'97*. IEEE, 1997, pp. 39–44.
- [3] T. Gries and V. Lutz, "8 - application of robotics in garment manufacturing," in *Automation in Garment Manufacturing*, ser. The Textile Institute Book Series. Woodhead Publishing, 2018, pp. 179–197.
- [4] D. Seita, A. Ganapathi, R. Hoque, M. Hwang, E. Cen, A. K. Tanwani, A. Balakrishna, B. Thananjeyan, J. Ichnowski, N. Jamali, K. Yamane, S. Iba, J. Canny, and K. Goldberg, "Deep imitation learning of sequential fabric smoothing from an algorithmic supervisor," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots. Syst.*, 2020, pp. 9651–9658.
- [5] J. Hietala, D. Blanco-Mulero, G. Alcan, and V. Kyrki, "Learning visual feedback control for dynamic cloth folding," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots. Syst.* IEEE, 2022, pp. 1455–1462.
- [6] F. Tokuda, A. Seino, A. Kobayashi, and K. Kosuge, "Cnn-based visual servoing for simultaneous positioning and flattening of soft fabric parts," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2023, pp. 748–754.
- [7] F. Tokuda, A. Seino, A. Kobayashi, K. Tang, and K. Kosuge, "Transformer driven visual servoing for fabric texture matching using dual-arm manipulator," *IEEE Robotics and Automation Letters*, vol. 11, no. 2, pp. 1522–1529, 2026.
- [8] M. Shetab-Bushehri, M. Aranda, Y. Mezouar, and E. Özgür, "Lattice-based shape tracking and servoing of elastic objects," *IEEE Trans. Robot.*, vol. 40, pp. 364–381, 2023.
- [9] T. Tang and M. Tomizuka, "Track deformable objects from point clouds with structure preserved registration," *Int. J. Robot. Res.*, vol. 41, no. 6, pp. 599–614, 2022.
- [10] X. Lin, Y. Wang, and D. Held, "Learning visible connectivity dynamics for cloth smoothing," in *CoRL*, 2021.
- [11] Z. Huang, X. Lin, and D. Held, "Self-supervised cloth reconstruction via action-conditioned cloth tracking," *arXiv preprint arXiv:2302.09502*, 2023.
- [12] A. Longhini, M. Büsching, B. P. Duisterhof, J. Lundell, J. Ichnowski, M. Björkman, and D. Kragic, "Cloth-splatting: 3d cloth state estimation from rgb supervision," *arXiv preprint arXiv:2501.01715*, 2025.
- [13] W. Wang, G. Li, M. Zamora, and S. Coros, "Trtm: Template-based reconstruction and target-oriented manipulation of crumpled cloths," in *Proc. IEEE Int. Conf. Robot. Autom.* IEEE, 2024, pp. 12 522–12 528.
- [14] E. Lo, X. Huang, K. Tang, A. Seino, F. Tokuda, and K. Kosuge, "Fabric flattening and alignment system using real-time mesh-based state estimation and visual servoing," in *Proc. IEEE Int. Conf. Mechatron. Autom.*, 2025, pp. 328–334.
- [15] K. Tang, F. Tokuda, A. Seino, A. Kobayashi, N. C. Tien, and K. Kosuge, "Time-scaling modeling and control of robotic sewing system," *IEEE/ASME Trans. Mechatron.*, vol. 29, no. 4, pp. 3166–3174, 2024.
- [16] W. Yan, A. Vangipuram, P. Abbeel, and L. Pinto, "Learning predictive representations for deformable objects using contrastive estimation," *CoRR*, vol. abs/2003.05436, 2020.
- [17] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.
- [18] T. Z. Zhao, J. Tompson, D. Driess, P. Florence, K. Ghasemipour, C. Finn, and A. Wahid, "Aloha unleashed: A simple recipe for robot dexterity," *arXiv preprint arXiv:2410.13126*, 2024.
- [19] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "o: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
- [20] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations," in *Proc. Robotics Sci. Syst.*, 2024.
- [21] Blender Online Community, "Blender – a 3d modelling and rendering package," <https://www.blender.org>, Amsterdam, 2025, [Online; accessed September 4, 2025].
- [22] N. Ravi, J. Reizenstein, D. Novotny, T. Gordon, W.-Y. Lo, J. Johnson, and G. Gkioxari, "Accelerating 3d deep learning with pytorch3d," *arXiv preprint arXiv:2007.08501*, 2020.
- [23] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, "Sam 2: Segment anything in images and videos," *arXiv preprint arXiv:2408.00714*, 2024.
- [24] P. Cignoni, M. Callieri, M. Corsini, M. Dellepiane, F. Ganovelli, and G. Ranzuglia, "MeshLab: an Open-Source Mesh Processing Tool," in *Eurographics Italian Chapter Conference*, V. Scarano, R. D. Chiara, and U. Erra, Eds. The Eurographics Association, 2008.
- [25] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zouitine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova *et al.*, "Lerobot: State-of-the-art machine learning for real-world robotics in pytorch," <https://github.com/huggingface/lerobot>, 2024.